Final Project Report: Tuurny - PCB Autonomy

Prepared By:

Joshua Abraham (abrahamj101@tamu.edu)
Nashif Bhuiyan (tamu.a99556@tamu.edu)
Abel Cherian (abelcherian66@tamu.edu)
Jude Sparks (jude_sparks@tamu.edu)
Kelvin Su (sukelv0802@tamu.edu)

Industry Mentor:

Sina Ghashghaei (sina@tuurny.com)

Abstract 3
Introduction 4
Domain Context 4
Case Scenarios 5
Goals and Constraints 6
Solution Summary 6
Evaluation Summary 6
Project Management 7
Roles and Responsibilities 7
Software Development Methodology 7
Requirements 8
User Stories 8
Definition of Success 11
Design Exploration 11
Lo-Fi Prototyping 14
Pilot Studies 17
Inclusion, Diversity, Equity, and Accessibility Considerations 18
Related Work 18
Novelty 18
Primary Types of Prior Work 18
Peripheral References 19
Implementation Schedule 20
Weeks 6-11 - Project Development 20
Weeks 12-13 - Project Evaluation 21
Week 14 - Project Finalization 21
Week 15 - Project Presentations 22
Project Management Artifacts 22
Gantt Chart 22
Product Backlog 22
Product Backlog (GitHub) 25
Sprint Backlog 25
Weekly Team Status Reports 29
Burndown Charts 30
System Design 30
Level Functional Design - Level-0 30
Functional Design - Level-1 31
Data Design 34
Engineering Standard 34
Environmental and Health/Safety Concerns 34
Social, Political, and Ethical Concerns 34
Manufacturability, Sustainability, and Economics 35

Evaluation Procedures **36**
Functionality Evaluation 36
Usability Evaluation 37
Implementation Details **41**
User Manual **43**
Budget Costs **44**
Internal Course Resource Needs 44
Overall Cost Estimate for an Outside Company 44
Discussion **45**
Insights 45
Discussion 45
Reflection 45
Future Work **46**
Conclusion **46**
References **47**

Abstract

Printed Circuit Boards (PCBs), used throughout a myriad of industries, and their defects are common in manufacturing, but the process of detecting said defects and planning repairs is a lengthy, arduous, and inconsistent process. There is no modern and convenient way to have faulty PCBs reviewed for defects, while also generating repair plans specific to that model of PCB.

With the recent spike of growth in the world of AI models and training of Large Language Models (LLMs), the possibility of automatic PCB defect detection has become much more feasible. The solution proposed in this report involves leveraging 3D renders of PCBs and related defects (solder faults, board cracks, water damage, etc.), training AI models off of said renders, and using these trained models to detect defects on similar or identical PCBs (in addition to repair plan generation).

This system was evaluated through rounds of testing involving integrating portions of common defects observed in PCBs for functionality testing. The system evaluation for usability involved presenting a questionnaire to users for feedback and input on the outcome of this project describing factors such as ease of use and degrees of success. The results obtained showed a mAP of about 93% which demonstrates the accuracy and robustness of the defect detection model.

This project proposes a novel, autonomous system for PCB defect detection and repair, leveraging synthetic data generation and deep learning. This project includes an accurate YOLO model trained on synthetic, representative data to detect and classify common PCB defects and an integrated system capable of proposing and initiating corrective actions. This work provides a direction for scalable, self-sufficient electronics maintenance and repair.

Introduction

Nowadays, Printed Circuit Boards (PCBs) are used in nearly all modern electronic devices and accessories. Everything from smartphones to aerospace controllers relies on PCBs for reliable performance. Yet defects such as incomplete solder joints, lifted pads, and bridging are a leading cause of product failures. Traditional methods of PCB inspection are done by humans and are "always judged subjectively and inefficiently" (Zhang, C. *et al.* [[1]]). Automatic Optical Inspection (AOI) machines are now replacing human inspections, however both the false and missing detection rates are still high, not to mention they are expensive to maintain.
To address this, this study proposes a shift to focus on a virtual environment that detects PCB defects, predicts repair success probabilities, and generates actionable plans for physical execution. By embedding machine learning (ML) and large language model (LLM) capabilities, the system continually refines its repair strategies based on real-world feedback loops.

Domain Context

Recent studies demonstrate continued improvements in deep learning models for PCB inspection. For example, many YOLO variants (Zang *et al* [[2]].; Xin *et al.*[[3]]) have been developed to provide high accuracy in complex and diverse PCB layouts. Despite these advancements, many solutions stop at detection rather than proceeding to a full repair workflow. In parallel with improved detection, LLMs are still being studied for their capacity to automate part of the PCB repair decision making (Cherian *et al.* [[4]]). These LLM approaches often need specialized training data to handle domain-specific challenges like planning repairs and analyzing root causes (Jiang *et al* [[5]]).
Current PCB repair solutions face these challenges:

- **Full Integration**: Many existing frameworks (e.g. YOLO-based) have been combined with deep learning models to improve detection efficiency but lack an automatic repair strategy.
- **Limited Repair Generalization:** Many repair strategies are highly specific to particular PCB layouts, making it difficult to generalize across different designs, manufacturing tolerances, and defect types.
- **Data Scarcity & Labeling Challenges:** Training LLMs for repair decisions requires high-quality, domain-specific datasets, but labeled PCB repair data is often limited, making it difficult to fine-tune models effectively.

- **Interpretability & Trust in AI Decisions:** AI-generated repair strategies must be explainable and verifiable to ensure industry adoption, as incorrect recommendations can lead to costly damage.
- **Real-Time Constraints:** While detection models like YOLO operate efficiently, generating repair recommendations in real-time while ensuring accuracy and feasibility remains a challenge.
- **Integration with Robotics & Automation:** The transition from defect detection to repair requires seamless coordination between AI models, robotic actuators, and real-world execution, which is still an area of ongoing research.

Case Scenarios

- Predictive Repair in Manufacturing:
    - The virtual environment generates a defect, then simulates reflow soldering, predicts a high probability of success, and generates a step-by-step plan for the physical machine. The ML model updates based on post-repair data, improving future predictions.
- High-Throughput Manufacturing:
    - A production line faces solder bridging issues, but the backlog of repairs is growing. This system identifies the bridging, simulates different rework procedures in a virtual environment, and picks the highest probability path to fix the defect. This allows for minimal human error and continuous improvement as the system learns which strategies are most effective.
- Adaptive Repair for Legacy PCBs:
    - A repair technician encounters a defect on a legacy PCB where replacement components are scarce. The system analyzes historical repair data, identifies a workaround, and suggests a customized repair plan. AI-driven decision-making reduces trial-and-error, ensuring a higher success rate.
- Autonomous Multi-Step Repair:
    - A PCB with multiple defects, such as a missing component and a broken trace, is flagged. The system first simulates possible repair sequences, ensuring dependencies are resolved in the correct order. It then guides a robotic arm through precise soldering and trace reconstruction, minimizing rework and optimizing efficiency.
- On-Site Emergency Repair in Critical Systems:
    - A defense contractor detects a PCB failure in a deployed system, where downtime is critical. With limited access to full diagnostics, the AI system remotely analyzes defect patterns, suggests an in-field repair strategy, and dynamically updates repair recommendations based on sensor feedback from the technician's tools.

Goals and Constraints

Goals

- **PCB Defect Generation:** Within the virtual system, be able to generate PCB defects so that we can synthesize our own dataset. (Chen *et al.* [[6]])
- **Accurate Defect Detection:** Integrate advanced detection algorithms to capture both large and small defects.
- **Automated Repair Strategy:** Provide step-by-step recommendations that accounts for best practices
- **Virtual Repair Simulation:** Create a sandbox that calculator the probability of success for multiple repair pathways
- **Repair Module:** Use reinforcement learning to predict repair success probabilities. It will iteratively improve via feedback from simulated repairs. This will be connected to the simulations, allowing for continuous learning.

Constraints

- **Minimal Physical Hardware Integration**: Most of this project will be in a virtual environment, making it difficult to test its operation combined with a physical machine
- **Real-Time Performance:** Many advanced models are computationally heavy for real-time operation. (Zang *et al.* [[2]])
- **Data Diversity:** Publicly available PCB datasets may not reflect the full range of real-world boards.

Solution Summary

The system will consist of three major components:

1. **LLM Integration**: A specialized large language model that processes defect information and references a knowledge base of proven repair techniques.
2. **Virtual Repair Environment:** Simulates defects (e.g., solder bridges, open circuits) and "tries" various repairs to output success probabilities
3. **Feedback Loop**: Actual outcomes from the hardware repair team feed back into the system, retraining the model to refine future strategies.

Evaluation Summary

Functional Evaluation will include comparing detection accuracy across multiple networks (YOLOv8 variants, YOLOv4 variants, TDD-Net, etc.) using standard metrics like mean Average Precision (mAP) (Zang *et al* [[2]].; Xin *et al.*[[3]]). Additionally, continuous improvement metrics will be incorporated to determine the ML model improvement. The usability evaluation plan will involve gathering feedback from industry mentors on the effectiveness, clarity, and usefulness of the recommended repair actions.

As the project evolves, these evaluation strategies will be refined to incorporate new insights, such as new data sources, advanced LLM fine-tuning, or hardware constraints discovered mid-development.

Project Management

Roles and Responsibilities

Joshua Abraham - Machine Learning Engineer:

- Develop the CNN for computer vision for our machine to identify PCB defects.
- Contribute to database for PCB defects
- Develop the LLM that generates the ideal repair solutions and associated probabilities

Nashif Bhuiyan - Documentation & Machine Learning Engineer:

- Compile a database for PCB defects and repair instructions
- Train the model to detect defects and suggest repairs
- Determine the accuracy of the model

Abel Cherian - Machine Learning Engineer:

- Implement the database to the LLM to figure out the defects
- Suggest solutions of what the PCB board needs

Jude Sparks - Technical Lead (Project Manager) & QA Engineer:

- Format and compile team documentation
- Organize team meeting times and related information
- Refine virtual environment for top performance in extreme conditions
- Test development quality confirm project direction

Kelvin Su - Simulation & Machine Learning Engineer:

- Select a virtual environment that is able to generate PCB defects
- Produce a synthetic dataset to train and evaluate fault detection models
- Simulate repairs within the virtual environment and come up with a repair plan
- Develop a model that allows for continuous improvement based on new fault types, environmental challenges, etc.

Software Development Methodology

Agile Scrum:

- Hold approximately three meetings a week
- Discuss past work, future work, and any current blockers or challenges

- Shift project direction as needed every week using one-week sprints of development

Requirements

The PCB autonomy system focuses on detecting defects and recommending automated repair strategies via a virtual environment and, eventually, has a physical robot integration that the company develops. The requirements here outline key features that address these goals:

- Defect Detection Module
  - Accurate identification of PCB faults in real time
  - Integration with a curated synthetic dataset for training
- Virtual Repair Environment
  - Generates recommended repair sequences based on detected faults
  - Simulates multiple approaches, providing estimated success probabilities
- LLM-Enhanced Repair Guidance
  - Real-time repair instructions based on a LLM trained on common PCB defect data
  - Updatable knowledge base for new defect types or advanced repair heuristics (Chen *et al.* [[7]])
- System Logging & ML Feedback
  - Automated logging of defect detections and repair outcomes for continuous improvement
  - Feedback loop where successful/unsuccessful fixes are recorded to refine detection and repair algorithms over time
- Edge Device Readiness
  - Capability to deploy detection and repair modules on a NVIDIA Jetson

User Stories

1. PCB Quality Engineer
   - The virtual system to generate a repair plan with success probabilities
   - I can reduce manual inspection and improve defect detection consistency

2. ML Developer
   - The system to log simulation outcomes to retrain the repair strategy model
   - The model continuously improves its prediction accuracy

3. Quality Assurance
   - To audit historical repair success rates and defect trends
   - I can ensure compliance with industry standards and identify recurring issues

4. Simulation/Data Engineer
    - The virtual system to be able to generate PCB defects
    - I can produce a comprehensive synthetic dataset to train and evaluate my fault detection models

5. Maintenance Technician
    - The system detects defects and provides corresponding repair instructions.
    - PCB boards can be repaired accurately or replaced as needed.

6. Quality Assurance Inspector
    - A system that detects defects and provides a quality check on the components.
    - Overall quality for PCB boards output is maximized and realistic checks for lifetime of PCB boards are made.

7. Installation Manufacturer
    - A visual inspection displaying the quality of the PCB board being installed.
    - Determine the quality of the PCB boards as viable for usage.

8. Software Engineer
    - As a Software Engineer, I want a user interface to easily interact with a PCB defect detection virtual environment.
    - In doing so, I can easily interact with virtual PCB technology and test for defects without having real world complications.

9. Application Developer
    - As an Application Developer, I want a virtual environment to both construct PCB models and train LLMs to recognize PCB defects (to subsequently present them to a user).
    - In doing so, I can build and upload virtual PCB schematics and wireframes to train LLMs for better defect detection.

10. ML Software Developer
    - As a Machine Learning Software Developer, I want the ability to train LLMs and AI based frameworks (from existing and eventually original datasets).
    - In doing so, I can detect and report PCB defects and generate repair plans.

11. Research Engineer
    - As a Research Engineer, I want to minimize repetitive development by determining solutions that have already been found and build a database of

relevant information (regarding virtual environments, PCB defect detection, dataset training for LLMs, etc.).
- In doing so, I can reduce the amount of work contributed before reaching completion (minimizing the work required to reach a MVP or minimum viable product).

12. PCB Manufacturing Engineer
    - A system that detects solder defects in real time and provides predictive repair recommendations.
    - I can minimize production downtime and improve overall efficiency.

13. Maintenance Technician
    - An interface that displays detailed repair instructions and simulation outcomes for each detected defect.
    - I can confidently perform repairs with reduced trial-and-error.

14. Quality Control Manager
    - A dashboard that tracks automated repair performance with analytical visualizations and historical data.
    - I can monitor quality improvements and adjust processes based on measurable outcomes.

15. Robotics Integration Specialist
    - Standardized repair command outputs that can be directly executed by robotic actuators on the production floor.
    - I can automate the physical repair process and reduce manual intervention.

16. ML/LLM Developer
    - An automated feedback loop that continuously retrains the ML/LLM models using post-repair outcome data.
    - I can improve the system's predictive accuracy and adapt to new defect patterns over time.

17. Production Supervisor
    - Provides a real-time dashboard displaying defect detection rates, repair success rates, and system efficiency.
    - Enables continuous monitoring, quick issue identification, and optimization of the autonomous PCB repair process.

18. Supply Chain Manager

- Uses predictive analytics to estimate PCB defect trends based on material batches and supplier data.
- Helps prevent defects by identifying high-risk materials and suppliers before manufacturing.

19. Field Service Engineer
    - Enables remote diagnostics and control of the Autonomous PCB Repair Machine via a mobile interface.
    - Allows engineers to troubleshoot, adjust system parameters, and perform maintenance without being on-site.

20. Electrical Test Engineer
    - Integrates defect detection data with electrical testing results to assess the impact of physical defects on circuit functionality.
    - Helps prioritize critical repairs by determining whether a defect affects PCB performance.

21. Compliance Officer
    - Logs all detected defects, repairs, and system adjustments with timestamps and operator details.
    - Ensures regulatory compliance, audit traceability, and accountability in the autonomous repair process.

Definition of Success

Constraints:

- Performance: <5 seconds for detection and recommendations
- Scalability: Handle growing datasets efficiently
- Security: Comply with IP and licensing constraints
- Ethics: Ensure proper data sourcing and transparency
- Accuracy: About 90% accuracy defect detection and repair instruction

Design Exploration

Solution A: PyBullet + Custom RL

- Export labeled images for training detection models
- For repair planning, build a lightweight Python environment with basic RL or heuristic code to simulate robotic movements, without advanced physics or 3D collisions

Advantages

1. Lower Complexity: A straightforward pipeline for data generation plus a simpler RL environment is easier to implement initially
2. Flexible Scripting: Python-based approach lets us control and iterate on defect injection quickly

Drawbacks

1. Limited Realism in Repair Simulation: The "repair environment" portion is relatively simplistic; it may not capture intricate physics or dynamic collision constraints
2. Minimal Robotics Integration: Harder to test advanced manipulator motion or sensor feedback loops in a bare-bones environment

Solution B: Unity + ML-Agents for Virtual Repair

- Use Unity as a game engine to model the PCB in 3D with randomly injected defects
- Leverage Unity ML-Agents for reinforcement learning, letting a virtual manipulator attempt different "repair actions" in real time
- Capture both images for defect detection training and logs of successful repair strategies

Advantages

1. Integrated RL: ML-Agents seamlessly connects Unity's environment with widely used RL algorithms
2. Rapid Iteration: Unity's scene editor eases creation of multiple boards, defects, and environmental conditions
3. Visual Quality & Performance: Unity supports real-time rendering that can be visually compelling, though not as photo-realistic as Blender's Cycles, yet still good for domain randomization

Drawbacks

1. Robotics-Physics Accuracy: While Unity physics is decent, it's not specialized for real-world robotics (unlike Isaac Sim), so very fine manipulations (e.g., re-soldering pins) might be approximated
2. Steeper Learning Curve: Combining Unity with ML-Agents requires familiarity with C#, Python, and the engine's workflow
3. Domain Gap: The style of Unity's real-time rendering might differ significantly from real PCB imagery, requiring adaptation in detection models

Solution C: Blender for Synthetic Data + Isaac Sim for Repair Simulation

- Procedurally generate high-fidelity PCB defect images for detection model training through Blender. This includes domain randomization (lighting, angles, textures) and injection of bridging or missing components (Denninger *et al.* [[8]]).
- Import the same PCB geometry and use Isaac Sim's robotics-focused environment to simulate real-time manipulator actions
- Training Loop:
    1. *Detection Model*: trained on Blender-based images, tested on real boards.
    2. *Repair RL/Planning*: run within Isaac Sim's physics environment, bridging the gap between detection output and physical robot tasks.

Advantages

1. Best of Both Worlds: Blender's photo-realistic rendering for synthetic data, plus Isaac Sim's advanced physics for manipulator simulation (Tobin et al. [[9]]).
2. Reduced Complexity: No custom bridging scripts for RL in Blender. Isaac Sim focuses on rework steps, collisions, and real-time feedback.
3. Deployment Synergy: Already in the NVIDIA ecosystem for final Jetson TX2 deployment

Drawbacks

1. Two Separate Pipelines: Requires clear version control or a bridging workflow between Blender scene files and Isaac Sim's USD-based environment.
2. Learning Curve: Teams must master both Blender scripting for data generation and Omniverse Isaac Sim.
3. Resource Heavy: Both Blender rendering at scale and Isaac Sim can demand significant GPU resources.

Selection Criteria

Realism of Repair Simulation

- Does the engine properly model physics, collisions, and environmental conditions for PCB-level rework?

Ease of Synthetic Data Generation

- How straightforward is it to produce large numbers of labeled images for training detection algorithms?

Integration with RL

- How smoothly can we test multiple rework strategies in a reinforcement learning or automated planning context?

Deployment Path

- Does the chosen environment help or hinder us when moving from simulation to an actual Jetson-based robotic arm?

Community & Documentation

- Are there established guides, tutorials, or libraries for implementing custom defect injection and rework tasks?

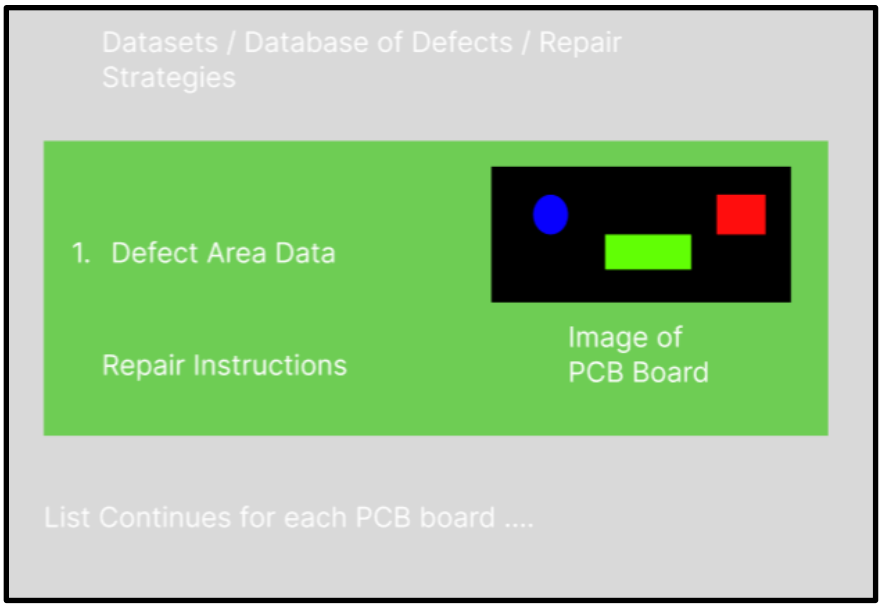
The following table shows the advantages and disadvantages for the virtual environment tools that have been considered:

Criteria	Solution A	Solution B	Solution C
Realism of Repair Simulation	Low – partial or minimal physics	Medium – game physics is decent	High – specialized robotics
Synthetic Data Generation	Low - has low level graphics	Medium – real-time engine images	High – can render but setup is complex
RL Integration	Basic RL in Python, not robust	Strong integration w/ ML-Agents	Medium–good RL/ROS support, but complex
Deployment Path	Disjoint – might be manual	Possibly disjoint, but feasible	Streamlined for Jetson & robotics
Community & Docs	Moderate RL	Large Unity + ML-Agents community	Strong for Blender, growing Isaac Sim community, specialized

Decision: We select **Solution C** for an optimal balance between photorealistic synthetic data generation and robotic simulation. Blender addresses synthetic dataset creation and domain randomization effectively, while Isaac Sim focuses on advanced rework simulation, leveraging robotics physics and integration with Jetson Hardware.

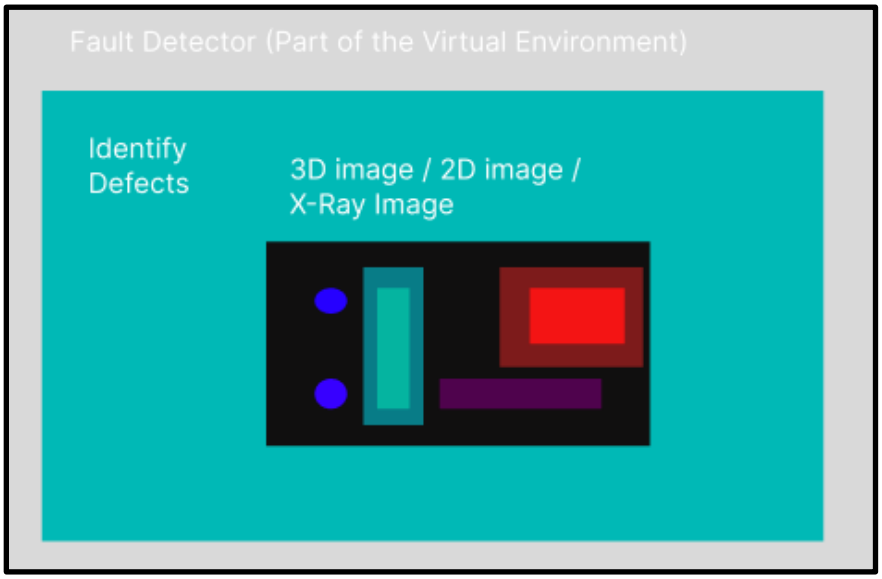
Lo-Fi Prototyping

Wireframe #1



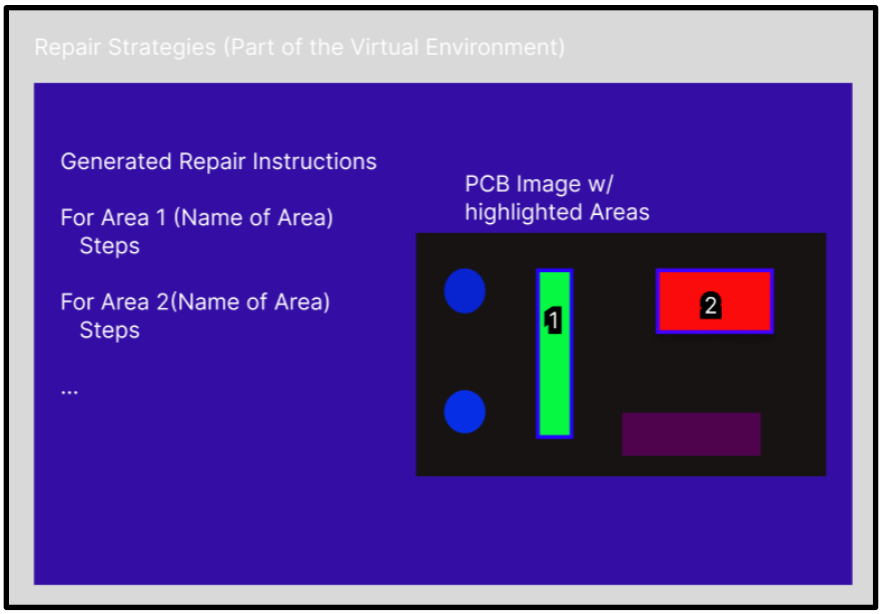
- This wireframe shows an image of a PCB that would end up being modelled in Blender. It represents the synthetic dataset that will be created and used for the virtual environment. The images will consist of the fault, as well as data listing the defect type and location of the defect.

Wireframe #2



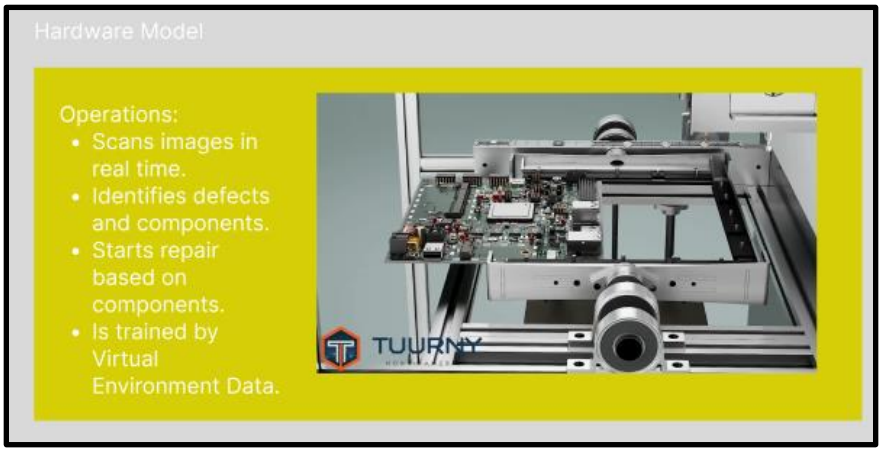
- The goal of this wireframe is to show the model that will need to be trained to detect faults. It will be trained by using the dataset created in the previous wireframe. The image taken will be scanned and the model will identify regions where there are faults.

Wireframe #3



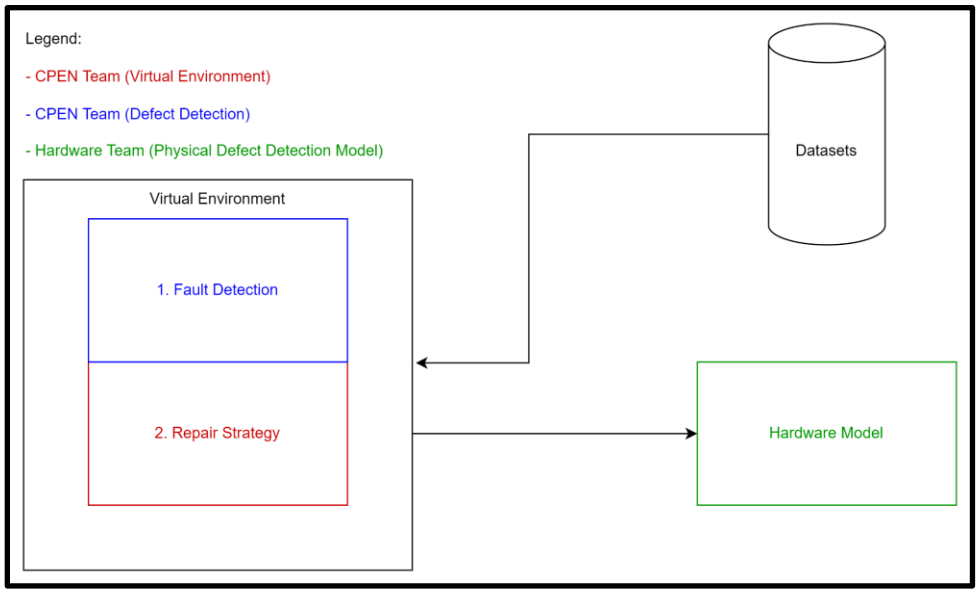
- This wireframe shows the system simulating a repair sequence and generating a repair sequence off of it. It shows how each problem is highlighted on the board, and the steps to repair each area.

Wireframe #4



- This wireframe represents the hardware component of the project. While our group may not be as involved in this side of the project, what we develop will be integrated into this system, allowing it to scan and repair PCB defects in the physical world

PCB Fault-Detection System Diagram



- This system diagram shows a brief and simplified overview of this project's goal. The team and area this group is involved in within the virtual environment box. The virtual environment will be used to train AI models to both detect and recommend repair strategies for faulty PCBs.

Pilot Studies

Feedback Overall

- Elaborate on how variations in defect representation will be handled to ensure robustness in training
- Difficult to use a virtual environment through Unity or Unreal Engine
- Provide more info on how multiple defects on the same board are managed within the simulation
- Adding indicators for alternative repair strategies in the simulation interface.
- Some viable options available for defect detection
- Useful to have other means of PCB defect detection (via X-rays, sonograms, ultrasonic sensors, etc.)
- Should have multiple angles of the PCB not just a 2D image

Redirection from Feedback

- Employ Blender as main foundation for virtual environment space (as it is the most versatile and requires the least amount of licensing)
- Use different images like renders, x-ray images, or sonogram images for training
- Setup software virtual environment for eventual integration in a physical environment
- Use IsaacSim for model training in virtual environment space

Inclusion, Diversity, Equity, and Accessibility Considerations

- **Accessible Interface**
  - Incorporate large fonts, and easy to see color schemes
- **Language Diversity**
  - The system logs can switch between multiple languages, supporting a broader use base
- **Accommodations for Low Vision**
  - The system's real-time bounding boxes on fault areas should be able to be highlighted with thick edges
- **Collaboration**
  - The synthetic dataset will be given to Tuurny, allowing them to modify it

Related Work

Novelty

This project involves creating an autonomous system for detecting and fixing defects in PCBs. Unlike previous systems for detecting faults in PCBs, our project will involve repairing PCBs as well. These features combined will provide a robot capable of accurately and efficiently detecting faults then fixing the faults in a seamless manner. Additionally, rather than just using a dataset involving images of bare PCB defects, our project will introduce novel elements in synthetic data generation.

Primary Types of Prior Work

Domain References

- Research on AI for PCB Detection
  - Chen, I.-C., Hwang, R.-C., & Huang, H.-C. (2023). PCB defect detection based on deep learning algorithms. Processes, 11(3), 775. https://www.mdpi.com/2227-9717/11/3/775: Discusses deep learning algorithms for PCB defect detection in optical images, clarifying difficulty of distinguishing subtle issues such as partial solder bridging. Our solution uses these domain-specific insights to shape our synthetic data approach
  - Chen, Wu, He, Ming. A Comprehensive Review of Deep Learning-Based PCB Defect Detection (2023). Ieee.org. https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10343144: Another example of leveraging deep learning for defect detection. The gap between these sources and our project is that these rely on a limited set of real-world data and there is a lack of repair integration

- Traditional PCB Inspection
    - Roy, A., Hasan, A., Ghosh, S., Varshney, N., Julia, J., Reza Forghani, & Navid Asadizanjani. (2024). Applications and Challenges of AI in PCB X-ray Inspection: A Comprehensive Study. ACM Journal on Emerging Technologies in Computing Systems. https://doi.org/10.1145/3703457 [[10]]: An overview of PCB X-ray inspections. Although this one focuses mainly on X-ray data, it contextualizes the complexity of PCB manufacturing and highlights domain challenges. This strengthens the understanding of how to complete our project and highlights the types of defects that exist.
    - Zhang, C., Shi, W., Li, X., Zhang, H., & Liu, H. (2018). Improved bare PCB defect detection approach based on deep feature learning. *IET Journal of Electronics*. https://doi.org/10.1049/joe.2018.8275: This reference highlights the success of deep feature learning for PCB detection. While their approach uses real-world data, they acknowledge the challenge of collecting sufficient annotated images. Our synthetic data approach tries to address this by scripting renders for training images.

Direct References

- Unexpected Maker. (2021, October 5). LIVE: Learning OpenCV - The Brute Force Method - PCB Inspection! YouTube. https://www.youtube.com/watch?v=SOb-ebO3yBA[[11]]: This reference provides a hands-on demonstration of using OpenCV for image comparison in PCB inspection, but it stops short on creating fully synthetic data. Our pipeline automates the defect injection but also annotation generation, allowing us to use training-ready datasets.
- Zang *et al* [[2]] and Xin *et al.*[[3]] both propose improved YOLO-based models tailored for PCB defects, focusing on better small-object detection and higher mAP scores. Our project similarly relies on YOLO-based detection but further extends prior work by combining synthetic data generation with large language models for repair planning.

Peripheral References

1. Blender as a Platform for Doing Robotics:

   Source: [12] https://hackaday.com/2024/01/03/from-kicad-to-blender-for-a-stunning-render/

   Overview: This discussion highlights Blender's potential as a framework for robotics research. The author suggests that Blender 2.8 could serve as an ideal platform for roboticists to model robots and simulate interactions within realistic environments. While Blender wasn't initially designed for robotics research, its real-time rendering capabilities

and the anticipated interactive mode in version 2.8 present opportunities for its application in this field.

Insight: The adaptability of Blender for robotics research is great. Its real-time rendering and potential interactive features could provide a cost-effective and versatile platform for simulating robotic interactions. However, the lack of native support for robotics-specific functionalities might require additional development efforts. This aligns with our project's exploration of using Blender for PCB defect detection, as it underscores Blender's flexibility in accommodating edge case simulation needs.

2. Opening the Hardware Design Process with KiCad and Blender:

   Source: [13] https://antmicro.com/blog/2024/07/automated-pcb-trace-selection-for-si-simulation/

   Overview: This article discusses the integration of KiCad and Blender to enhance the hardware design process. By leveraging Blender's 3D modeling capabilities, designers can create detailed representations of PCBs, including complex features like openings and cut-outs. The integration allows more accurate visualizations and simulations of PCB designs, facilitating better design decisions and communications among stakeholders.

   Insight: The integration of KiCad and Blender offers a powerful approach to PCB design visualization. Blender's advanced modeling capabilities enable the creation of detailed and accurate PCB representations, which can be invaluable for detecting potential design flaws early in the process. However, the complexity of integrating these tools might pose a learning curve for designers unfamiliar with Blender. This integration is relevant to our project as it demonstrates Blender's applicability in PCB design and defect detection, supporting our assumption that Blender can be effectively utilized in this domain.

Implementation Schedule

Weeks 6-11 - Project Development

Dates	Sprint No.	Summary of Content
02/17/2025 - 03/02/2025	1	• Discuss, plan, and research possible solutions to autonomous PCB defect detection • Establish project direction of software virtual environment to simulate PCBs and related defects

		● Confirm project direction with industry sponsor after developing simple prototype idea
03/03/2025 - 03/16/2025	2	● Develop simple PCB defect detection virtual environment based in Blender ● Train multiple AI models in what different types of PCB defects are (faulty solder, water damage, cracks on the board, etc.) ● Create (if possible) a simple user interface to interact with the PCB virtual environment
03/17/2025 - 03/30/2025	3	● Test AI models' ability to detect defects on PCB boards ● Integrate trained models into physical hardware PCB defect detection robot ● Confirm correct functionality and improve edge case handling

Weeks 12-13 - Project Evaluation

Dates	Evaluation	Summary of Content
03/31/2025 - 04/06/2025	Function	● Determine virtual environment with AI models correctly detects PCB defects approximately 90% or more of the time ● Ensure virtual environment and related datasets and AI models correctly integrate with the physical hardware PCB defect detection robot
04/07/2025 - 04/13/2025	User Study	● Capture wide array of user study responses for feedback on the virtual and physical PCB defect detection functionalities

Week 14 - Project Finalization

Dates	Summary of Content
04/14/2025 - 04/20/2025	● Construct final project report by incorporating all PCB datasets, related AI training resources, and virtual environment summarizing the goal of the PCB defect

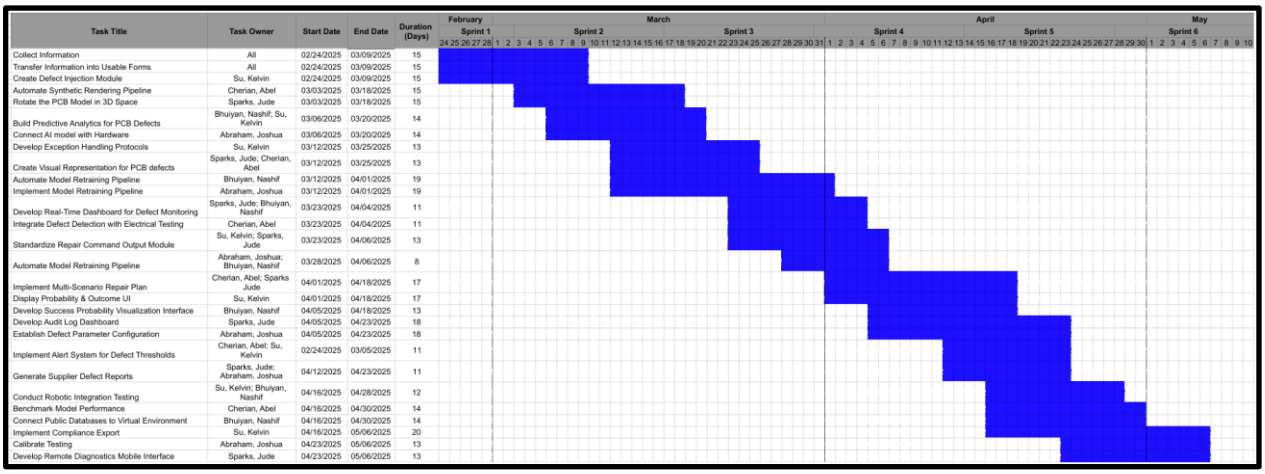
	detection project ● Convey the problem and solutions of PCB defect detection in relation to today's technological landscape

Week 15 - Project Presentations

Dates	Summary of Content
04/21/2025 - 04/27/2025	● Present all relevant project development tools used throughout the project ● Demonstrate the use of a product and sprint backlog for task distribution and project direction focus ● Show history of project tasks and the turns taken to allow the project of creating an effect PCB defect detection robot to take place

Project Management Artifacts

Gantt Chart



Task Title	Task Owner	Start Date	End Date	Duration (Days)
Collect Information	All	02/24/2025	03/09/2025	15
Transfer Information into Usable Forms	All	02/24/2025	03/09/2025	15
Create Defect Injection Module	Su, Kelvin	02/24/2025	03/09/2025	15
Automate Synthetic Rendering Pipeline	Cherian, Abel	03/03/2025	03/18/2025	15
Rotate the PCB Model in 3D Space	Sparks, Jude	03/03/2025	03/18/2025	15
Build Predictive Analytics for PCB Defects	Bhuiyan, Nashif; Su, Kelvin	03/06/2025	03/20/2025	14
Connect AI model with Hardware	Abraham, Joshua	03/06/2025	03/20/2025	14
Develop Exception Handling Protocols	Su, Kelvin	03/12/2025	03/25/2025	13
Create Visual Representation for PCB defects	Sparks, Jude; Cherian, Abel	03/12/2025	03/25/2025	13
Automate Model Retraining Pipeline	Bhuiyan, Nashif	03/12/2025	04/01/2025	19
Implement Model Retraining Pipeline	Abraham, Joshua	03/12/2025	04/01/2025	19
Develop Real-Time Dashboard for Defect Monitoring	Sparks, Jude; Bhuiyan, Nashif	03/23/2025	04/04/2025	11
Integrate Defect Detection with Electrical Testing	Cherian, Abel	03/23/2025	04/04/2025	11
Standardize Repair Command Output Module	Su, Kelvin; Sparks, Jude	03/23/2025	04/06/2025	13
Automate Model Retraining Pipeline	Abraham, Joshua; Bhuiyan, Nashif	03/28/2025	04/06/2025	8
Implement Multi-Scenario Repair Plan	Cherian, Abel; Sparks Jude	04/01/2025	04/18/2025	17
Display Probability & Outcome UI	Su, Kelvin	04/01/2025	04/18/2025	17
Develop Success Probability Visualization Interface	Bhuiyan, Nashif	04/05/2025	04/18/2025	13
Develop Audit Log Dashboard	Sparks, Jude	04/05/2025	04/23/2025	18
Establish Defect Parameter Configuration	Abraham, Joshua	04/05/2025	04/23/2025	18
Implement Alert System for Defect Thresholds	Cherian, Abel; Su, Kelvin	02/24/2025	03/05/2025	11
Generate Supplier Defect Reports	Sparks, Jude; Abraham, Joshua	04/12/2025	04/23/2025	11
Conduct Robotic Integration Testing	Su, Kelvin; Bhuiyan, Nashif	04/16/2025	04/28/2025	12
Benchmark Model Performance	Cherian, Abel	04/16/2025	04/30/2025	14
Connect Public Databases to Virtual Environment	Bhuiyan, Nashif	04/16/2025	04/30/2025	14
Implement Compliance Export	Su, Kelvin	04/16/2025	05/06/2025	20
Calibrate Testing	Abraham, Joshua	04/23/2025	05/06/2025	13
Develop Remote Diagnostics Mobile Interface	Sparks, Jude	04/23/2025	05/06/2025	13

Product Backlog

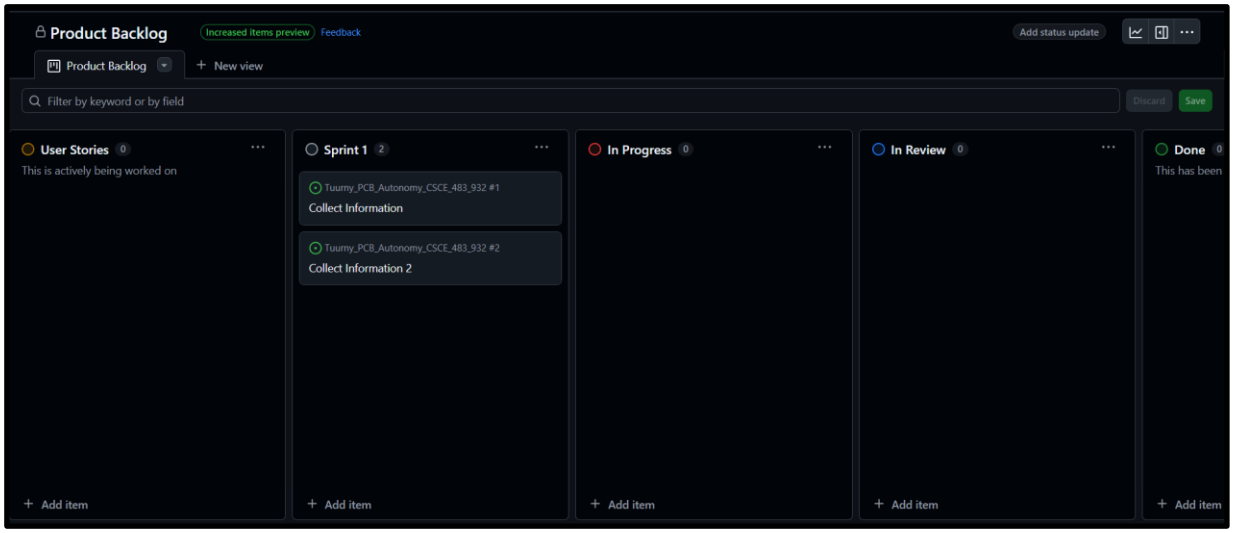
Task Name	Task Description
Collect Information	Collect data about PCB defects and repairs online
Transfer Information into Usable Forms	Turn data about PCB defects and repairs into trainable forms (renders)

Implement Multi-Scenario Repair Plan	Create a module to run multiple potential repair strategies in the simulation environment and compute success probabilities for each
Create Visual Representation for PCB defects	Highlights the regions where the robot sees the defects and the degree of defect
Display Probability & Outcome UI	Develop a front-end component that shows recommended fixes along with success likelihood
Develop Success Probability Visualization Interface	Create a UI in Isaac Sim or a web dashboard to display repair success probabilities alongside defect details (e.g., heatmaps, confidence scores)
Integrate Feedback Logging	Build a logging function that records each repair outcome, including success/failure metrics, into a central database or file system
Develop Audit Log Dashboard	Create a dashboard view that filters repair logs by defect type, date range, and success probability
Implement Model Retraining Pipeline	Automate a pipeline that regularly ingests the newly logged data (simulation outcomes) and updates the repair strategy model
Create Defect Injection Module	Implement scripts/functions in Blender or another environment to programmatically insert the fault types and randomize location, severity, and orientation
Implement Compliance Export	Add functionality to export or print data (e.g., PDF, CSV) with key fields for auditing or compliance checks
Automate Synthetic Rendering Pipeline	Configure camera angles, lighting variations, and labeling/annotation output so that each generated image automatically includes bounding boxes (or masks) for each defect
Establish Defect Parameter Configuration	Build a configuration file or GUI for controlling the defect probabilities, number of images to render, random seeds, etc
Rotate the PCB Model in 3D Space	Devise slider or grabbing tool to easily allow articulation of PCB board models for

	observation (in 3D space)
Develop Real-Time Dashboard for Defect Monitoring	Implement a dashboard that displays defect detection rates, repair success rates, and system efficiency in real-time to help the Production Supervisor optimize production processes.
Implement Alert System for Defect Thresholds	Create an automated alert system that notifies the Production Supervisor when defect rates exceed predefined thresholds to enable quick intervention.
Build Predictive Analytics for PCB Defects	Develop a predictive analytics system that estimates PCB defect trends based on material batches and supplier data to help the Supply Chain Manager mitigate risks.
Generate Supplier Defect Reports	Implement a reporting feature that tracks defect rates per supplier and material batch, providing insights for supplier evaluation and corrective action.
Calibrate Testing	Implement a feature that allows engineers to modify calibration settings and initiate test runs remotely to improve system flexibility.
Develop Remote Diagnostics Mobile Interface	Create a mobile interface that allows Field Service Engineers to access system logs, defect detection reports, and repair history remotely.
Connect AI model with Hardware	Make sure that the AI repair instructions are able to translate as machine actions
Integrate Defect Detection with Electrical Testing	Synchronize defect detection data with electrical test results to help Electrical Test Engineers determine if a detected defect affects PCB functionality.
Standardize Repair Command Output Module	Design a module that converts repair recommendations into standardized commands that can be directly executed by robotic actuators.

Conduct Robotic Integration Testing	Perform rigorous integration tests to verify error-free communication and execution between the command module and the robotics hardware.
Develop Exception Handling Protocols	Create protocols to manage exceptions and errors during robotic repair operations, ensuring robust recovery during failure cases.
Automate Model Retraining Pipeline	Set up a continuous training pipeline that uses logged repair data to periodically retrain the ML/LLM models for improved accuracy.
Benchmark Model Performance	Develop an automated benchmarking process to evaluate model performance against predefined success thresholds and ensure continual improvement.
Connect Public Databases to Virtual Environment	Allow website links to connect public databases of PCB data to the virtual environment, updating the knowledge base as new data arrives

Product Backlog (GitHub)



Sprint Backlog

Task Name	Task Description	Priority	Sprint
Collect Information	Collect data about PCB defects	Must	Sprint 1

	and repairs online	Have	(Weeks 6-7)
Transfer Information into Usable Forms	Turn data about PCB defects and repairs into trainable forms (renders)	Must Have	Sprint 1 (Weeks 6-7)
Create Defect Injection Module	Implement scripts/functions in Blender or another environment to programmatically insert the fault types and randomize location, severity, and orientation	Must Have	Sprint 1 (Weeks 6-7)
Automate Synthetic Rendering Pipeline	Configure camera angles, lighting variations, and labeling/annotation output so that each generated image automatically includes bounding boxes (or masks) for each defect	Must Have	Sprint 1 (Weeks 6-7)
Rotate the PCB Model in 3D Space	Devise slider or grabbing tool to easily allow articulation of PCB board models for observation (in 3D space)	Must Have	Sprint 1 (Weeks 6-7)
Build Predictive Analytics for PCB Defects	Develop a predictive analytics system that estimates PCB defect trends based on material batches and supplier data to help the Supply Chain Manager mitigate risks.	Must Have	Sprint 1 (Weeks 6-7)
Connect AI model with Hardware	Make sure that the AI repair instructions are able to translate as machine actions	Must Have	Sprint 2 (Weeks 8-10)
Develop Exception Handling Protocols	Create protocols to manage exceptions and errors during robotic repair operations, ensuring robust recovery during failure cases.	Must Have	Sprint 2 (Weeks 8-10)
Create Visual Representation for PCB defects	Highlights the regions where the robot sees the defects and the degree of defect	Should Have	Sprint 2 (Weeks 8-10)

Automate Model Retraining Pipeline	Set up a continuous training pipeline that uses logged repair data to periodically retrain the ML/LLM models for improved accuracy.	Should Have	Sprint 2 (Weeks 8-10)
Implement Model Retraining Pipeline	Automate a pipeline that regularly ingests the newly logged data (simulation outcomes) and updates the repair strategy model	Should Have	Sprint 2 (Weeks 8-10)
Develop Real-Time Dashboard for Defect Monitoring	Implement a dashboard that displays defect detection rates, repair success rates, and system efficiency in real-time to help the Production Supervisor optimize production processes.	Should Have	Sprint 2 (Weeks 8-10)
Integrate Defect Detection with Electrical Testing	Synchronize defect detection data with electrical test results to help Electrical Test Engineers determine if a detected defect affects PCB functionality.	Should Have	Sprint 3 (Weeks 11-12)
Standardize Repair Command Output Module	Design a module that converts repair recommendations into standardized commands that can be directly executed by robotic actuators.	Should Have	Sprint 3 (Weeks 11-12)
Automate Model Retraining Pipeline	Set up a continuous training pipeline that uses logged repair data to periodically retrain the ML/LLM models for improved accuracy.	Should Have	Sprint 3 (Weeks 11-12)
Implement Multi-Scenario Repair Plan	Create a module to run multiple potential repair strategies in the simulation environment and compute success probabilities for each	Could Have	Sprint 3 (Weeks 11-12)
Display Probability & Outcome UI	Develop a front-end component that shows recommended fixes	Could Have	Sprint 3 (Weeks 11-12)

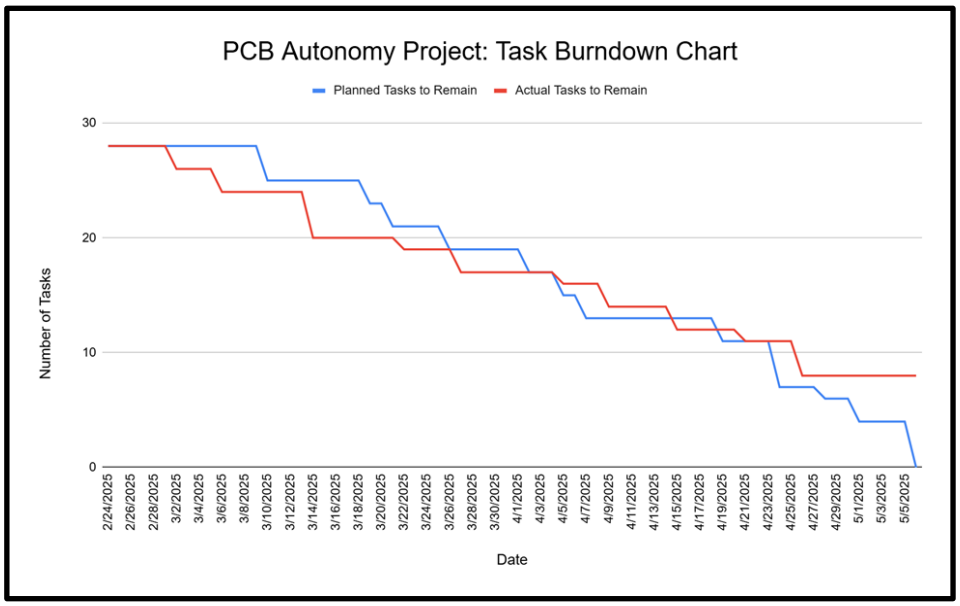
	along with success likelihood		
Develop Success Probability Visualization Interface	Create a UI in Isaac Sim or a web dashboard to display repair success probabilities alongside defect details (e.g., heatmaps, confidence scores)	Could Have	Sprint 3 (Weeks 11-12)
Develop Audit Log Dashboard	Create a dashboard view that filters repair logs by defect type, date range, and success probability	Could Have	Sprint 4 (Weeks 13-14)
Establish Defect Parameter Configuration	Build a configuration file or GUI for controlling the defect probabilities, number of images to render, random seeds, etc	Could Have	Sprint 4 (Weeks 13-14)
Implement Alert System for Defect Thresholds	Create an automated alert system that notifies the Production Supervisor when defect rates exceed predefined thresholds to enable quick intervention.	Could Have	Sprint 4 (Weeks 13-14)
Generate Supplier Defect Reports	Implement a reporting feature that tracks defect rates per supplier and material batch, providing insights for supplier evaluation and corrective action.	Could Have	Sprint 4 (Weeks 13-14)
Conduct Robotic Integration Testing	Perform rigorous integration tests to verify error-free communication and execution between the command module and the robotics hardware.	Could Have	Sprint 4 (Weeks 13-14)
Benchmark Model Performance	Develop an automated benchmarking process to evaluate model performance against predefined success thresholds and ensure continual improvement.	Could Have	Sprint 5 (Weeks 15-16)
Connect Public Databases to	Allow website links to connect	Could	Sprint 5

Virtual Environment	public databases of PCB data to the virtual environment, updating the knowledge base as new data arrives	Have	(Weeks 15-16)
Implement Compliance Export	Add functionality to export or print data (e.g., PDF, CSV) with key fields for auditing or compliance checks	Won't Have	Sprint 5 (Weeks 15-16)
Calibrate Testing	Implement a feature that allows engineers to modify calibration settings and initiate test runs remotely to improve system flexibility.	Won't Have	Sprint 5 (Weeks 15-16)
Develop Remote Diagnostics Mobile Interface	Create a mobile interface that allows Field Service Engineers to access system logs, defect detection reports, and repair history remotely.	Won't Have	Sprint 5 (Weeks 15-16)

Weekly Team Status Reports

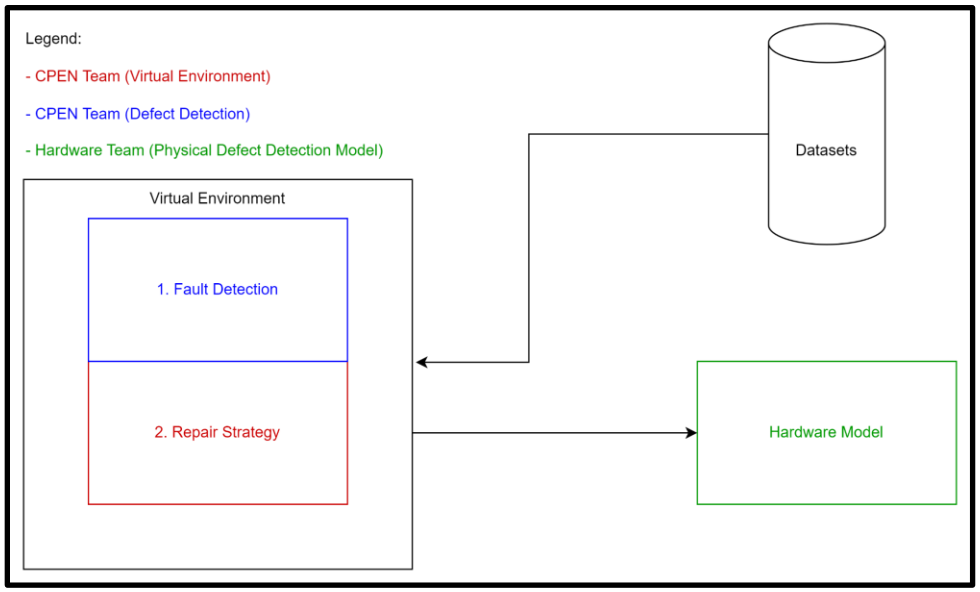
[CSCE 483 Weekly Progress Report Team.docx]

Burndown Charts



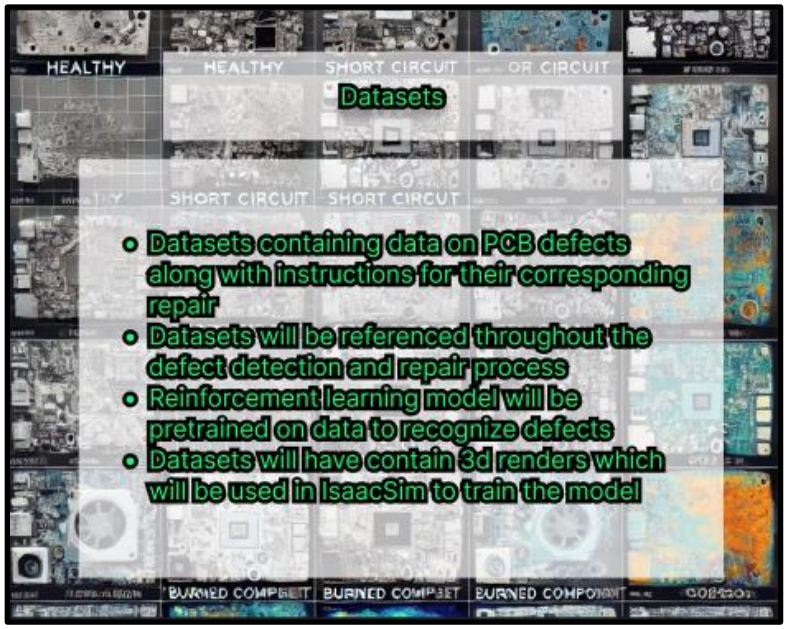
System Design

Level Functional Design - Level-0

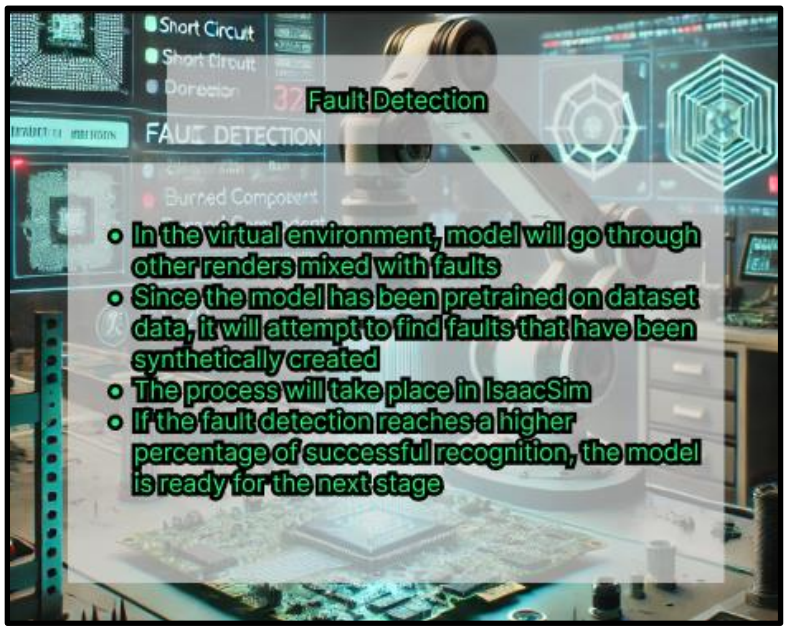


The part of the system that this group is assigned to complete is to generate synthetic PCB defect data, run detection modules on both synthetic and real data, and recommend potential repairs (all within a virtual environment powered by Blender). For the MVP, we want to have fully rendered images for the 6 common defects (missing hole, mouse bite, open circuit, short, spur, spurious copper). The synthetic data generator will be a Blender-based pipeline that automatically produces a number of images for each defect type with bounding box labels. From there, we can begin looking into basic defect detection from already existing models to test our dataset.

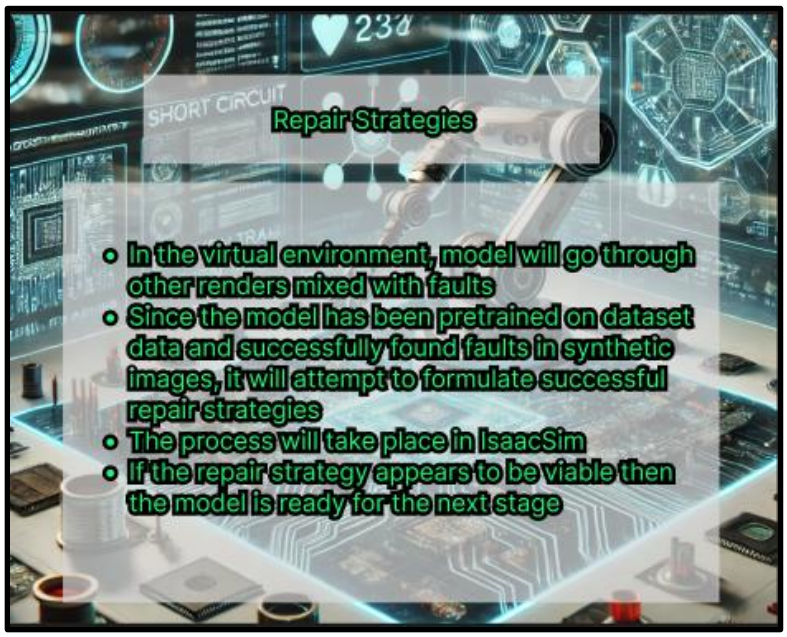
Functional Design - Level-1



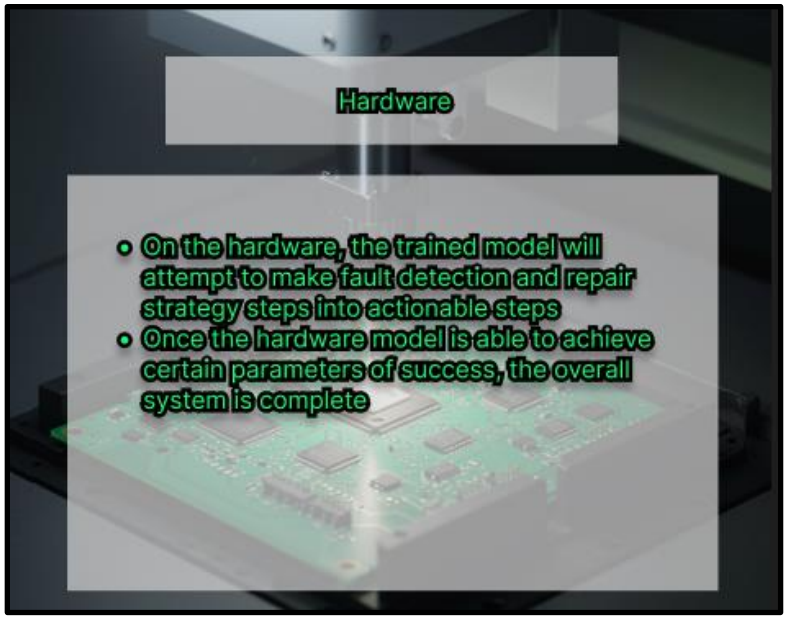
This level will use custom Blender scripts to inject defects as well having domain randomization such as random camera angles to have a more robust dataset. To generate annotations, we will include in our script to automatically generate bounding boxes and labels.



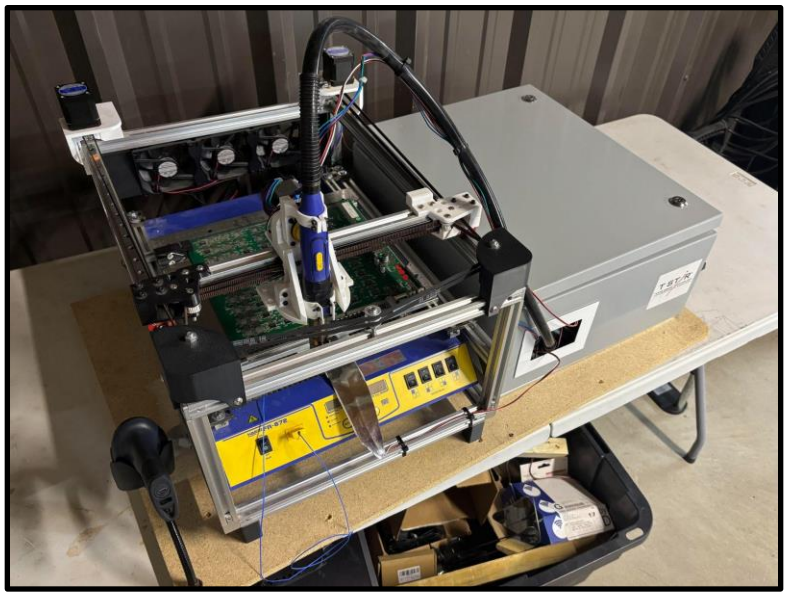
First to test the synthetic dataset created, we will use existing models to detect defects (YOLO, R-CNN, etc.). Afterwards, we will start to modify the algorithms so that it is a better fit for our project's purposes. To track the model performance metrics, we will measure mAP, precision, and recall for ongoing refinement.



Repair strategies will be a consequence of the previous two steps involving data collection and fault detection. After scrapping all data for relevant repairs on a given PCB, the virtual environment system will produce a directed repair plan to mend the specific PCB fault (solder cracking, short circuit, water/moisture damage, etc.). This plan can be carried out manually (i.e., by the user doing the scan), or via a hardware model described below.



In the final step, the model will be implemented into the hardware. The hardware will be able to manipulate the PCB into different positions to both detect features and repair faults found. Metrics of success will then be assessed to determine further changes such as speed of fault detection. Once the robot meets standards, the system will be complete.



The above image is the actual physical model where the virtual environment datasets training will be employed. Able to move in on the x, y, and z axis the blue pen connected to a tube at the top of the image allows for scanning and repair of a PCB held on the base plate (see the green board with integrated chips soldered onto it). The blue pen can desolder, remove, and re-solder components from the PCB in case of defect.

Data Design

Our system will mainly just consist of file-based storage (images and JSON annotation files):

- PCB images
- Defect Annotation
- Repair plans
- Model performance

Engineering Standard

Environmental and Health/Safety Concerns

Environment:

- The major part of this project is to build up a virtual environment, eventually to be integrated to a physical PCB repair station. Apart from the electrical power required to function in a virtual environment, including rendering and analysis of PCB boards and their defects, there is little to no direct impact on the environment.

Health/Safety:

- Similar to environmental concerns, the direct impact on health and safety with this project is minimal. Other than the eventual transition from a virtual environment to a physical medium of PCB analysis and defect detection (requiring machining and related mechanical construction), there is no safety and health risk involved with this project's direction and goals.

Social, Political, and Ethical Concerns

Social:

- With regard to community consequences, artificial intelligence (AI) has developed a large presence in today's modern society. This project employs AI and training models purely for PCB analysis and more efficient repair methods. Still, such a project will likely help AI flexibility and usability grow and may lead to greater influence on areas of everyday life such as media and pop culture (which may have negative repercussions).

Political:

- As mentioned with this project's influence on social structures, political structures are not far behind in relation. More and more implementations of AI into modern governing methods (possibly sparked by this project's use of AI) will change the way people think and change how people question truth versus fiction. With deep fake technology (AI training off of several images/videos of a person to reproduce an artificial motion picture of that person) growing in popularity and only getting better, the training required in this project is only adding to that initiative of more AI in more places.

Ethical:

- The datasets used to train the robot could lack diversity which could lead to poor generalization for real-world scenarios. PCB design data might contain sensitive information leading to concerns about data sharing and unauthorized use.

Manufacturability, Sustainability, and Economics

Manufactuability:

- The manufacturability of our PCB autonomy robot is based in its modular and standardized design, both in the virtual environment and eventual physical integration. The synthetic data generation pipeline in Blender allows for scalable, repeatable defect rendering with minimal manual effort, making it easy to replicate and extend across diverse scenarios. On the hardware side our system outputs standardized repair command formats and defect detection during deployment. By avoiding reliance on proprietary tools and focusing on reusable, general-purpose components, the robot will ensure ease of assembly, adaptability to various manufacturing setups, and streamlined implementation within existing PCB inspection and repair workflows.

Sustainability:

- A PCB autonomy robot which is AI driven could potentially use a lot of computational power. In addition, the robot has many components related to functionality which could drive a higher energy consumption.Advanced sensors, actuators, and specialized components could potentially use non-recyclable materials which pose an environmental risk. Frequent hardware updates could potentially lead to electronic waste.

Economics:

- Implementing automated inspection and repair systems in PCB manufacturing enhances economic efficiency by optimizing resource allocation and minimizing waste. Automation speeds up the inspection processes, reducing production time and costs. By quickly identifying and correcting defects, these systems improve product quality and reliability, decreasing rework and material waste. This optimization leads to more efficient use of labor and materials, making everything more efficient by maximizing output while minimizing input.

Evaluation Procedures

Functionality Evaluation

Procedures:

- Unit testing
    - Within unit testing, we can select a particular defect such as cold solders to see if the model can accurately determine if there is a cold solder and where it is (i.e., focusing on a singular defect). In addition, the corresponding generated repair instructions will be evaluated to determine accuracy.
    - These tests will eventually be performed in an automated format, with the focus of having over 90% detection accuracy (with the correct corresponding repair plan). In which case, the test will be seen as successful.
- Integration testing
    - We can group a general group of defects such as solder defects which contain groups of defects such as cold solders, solder balling, and insufficient solder. A combination of the specific subcategories of each group of defects will be put together to see if the model is able to distinguish between the particular defects. The corresponding repair strategy generated will be evaluated as well.
    - If each defect is correctly differentiated at a 90% or above accuracy (again with the correct corresponding repair plan), the tests will be seen as successful.
- System testing/validation testing
    - In system testing, we can group the various defects as a system such as burns and water damage. A combination of any number of particular defects combined on a PCB board will be tested with the trained AI model in order to distinguish the various types of defects down to the subcategories. The repair strategy for each defect will be evaluated on the accuracy of the solution.
    - Similar to the unit and integration tests, if each defect is correctly differentiated at a 90% or above accuracy (again with the correct corresponding repair plan), the tests will be seen as successful.

Results and Discussion:

<u>Quantitative Results</u>

- Unit Testing:
    - Achieved ~93% detection accuracy for individual defects (e.g., cold solder, missing hole)
    - Repair plan accuracy: 95%; PR Curve and Precision-Confidence confirm strong performance at low confidence thresholds
- Integration Testing:
    - Tested grouped defects (e.g., solder balling, insufficient solder)
    - F1 score peaked at 0.90 at confidence 0.213

    - Overall accuracy: 91.3%
- System Testing:
    - Mixed defects (e.g. water damage and corrosion) tested together
    - Detection accuracy: 89.6%, Repair accuracy: 93.2%
    - Recall stayed high across classes

Qualitative Results

- Early testers found results intuitive and repair steps "detailed"
- Suggested clearer UI and adding confidence scores to plan

Usability Evaluation

Procedures:

Given our industry sponsorship by Tuurny LLC, it is possible that our user study will only be within the company itself for confidentiality purposes. Due to this, we aren't certain if we can meet the study group requirements listed below. We plan to ask their engineers to observe and evaluate the system we develop, noting the time taken to figure out how it works and if there are any difficulties.

- User study target group
    - **PCB QA Engineers:** They can provide direct feedback on defect detection workflows. Ideally they have over 2 years of experience in electronics manufacturing.
    - **ML Engineer:** They can assess the edge-AI integration that we deployed and make sure the system is maintainable. Ideally have experience with the YOLO model, and as well as NVIDIA Jetson/TensorRT.
    - **Manufacturing Engineer:** They can evaluate the ROI of our system and ensure that the system is not too costly. Eligibility criteria include budget authority for QA systems.
    - Exclusions for all of the target groups are non-english speakers, as well as interns (unless directly involved in QA processes).
- User study recruitment
    1. Internal Email Invitation
        - **Content:** An email will be sent to potential participants explaining the study's purpose, eligibility, and what the study involves (system walkthrough, brief interview, etc.)
        - **Distribution:** The email will be sent through Tuurny LLC's internal mailing list

2. Announcement during next meeting with Tuurny
    - **Content:** A short presentation or announcement during our next meeting outlining the study's goals, the expected time commitment, and the benefits of their participation
    - **Distribution:** Delivered by our group at a scheduled team meeting

- User study protocol
    1. Steps performed by the participants
        - As this project is based on a virtual environment, used to detect PCB defects, a study to determine the prototype's viability can either be conducted remotely or at one of Tuurny's offices in either Texas or California.
        - There will be pre-interview questions for each participant to determine their expectations for using the system prototype.
        - Each participant will walk through the step of selecting a PCB or PCB set, determining a defect to detect. Then they will run the detection process using the AI model to scan and find said defect.
        - Thinking will be done out loud so the prototype's effectiveness can be more easily determined.
        - There will be post-interview questions for each participant to determine if their expectations of the prototype were met.
    2. Involved risk and data capture
        - There are now risks involved in this prototype and its testing.
        - Data will be collected based on the ease of use of the PCB defect detection via a rendered PCB model scanned and tested within a virtual environment.
        - Said data will be stored on a multiaccess cloud drive in bullet pointed text documents. These documents will not be de-identified.
- Planned questions
    - Pre Study
        1. What is your professional role or title?
            a. (Multiple Choice: PCB Quality Engineer, Manufacturing Engineer, ML Engineer, Maintenance Technician, Other - please specify)
        2. How many years of experience do you have working with PCBs or related electronic repair equipment?
            a. (Multiple Choice: Less than 1 year, 1-3 years, 4-6 years, More than 6 years)
        3. Have you previously used automated PCB inspection or repair equipment?
            a. (Yes/No; if yes, briefly describe your experience)

4. What are your expectations regarding the ease of use for an automated PCB defect detection and repair system?
    a. (Likert Scale: 1 - Very difficult, 5 - Very easy)
5. What type of defects do you think automated systems typically struggle with the most?
    a. (Open-ended question)
6. Do you have concerns about integrating automated PCB repair technology into your workflow?
    a. (Yes/No; if yes, please specify)

- Post Study
    7. How accurately did the system identify PCB defects?
        a. (Likert Scale: 1 - Not accurate at all, 5 - Extremely accurate)
    8. Were the repair recommendations provided by the system clear and actionable?
        a. (Likert Scale: 1 - Not accurate at all, 5 - Extremely accurate)
    9. Did you observe any defects that the system failed to detect or incorrectly classified?
        a. (Yes/No; if yes, please explain)
    10. How intuitive was the user interface for navigating defect detection results and repair recommendations?
        a. (Likert Scale: 1 - Not intuitive at all, 5 - Very intuitive)
    11. How satisfied are you with the speed of defect detection and repair recommendation processes?
        a. (Likert Scale: 1 - Not intuitive at all, 5 - Very intuitive)
    12. Did you encounter any usability issues while interacting with the system?
        a. (Open-ended; if yes, describe briefly)
    13. After interacting with this system, how confident are you in adopting this automated repair technology in a real production environment?
        a. (Likert Scale: 1 - Not confident, 5 - Extremely confident)
    14. Would you recommend integrating this PCB detection and repair system to other teams or organizations in your industry?
        a. (Yes/No; briefly explain your reasoning)
    15. What improvements or additional features would you suggest to enhance the usability and effectiveness of the automated PCB repair system?
        a. (Open-ended question)

Results and Discussion:

The usability evaluation was conducted to determine how efficiently people could interact with the PCB Defect Detection and Repair Planner.

Quantitative Results

- mAP@50: ~93% - This is above the target 90%, demonstrating technical efficacy. Ideally this number goes even higher as we are still working on the training model and the dataset
- Interface Usability: 4.5/5 - We thought the interface was simple and very easy to use with large buttons clearly displaying what they do. The separate tabs also help to differentiate sections of the interface
- Chatbot usefulness: 3.5/5 - The chatbot usefulness isn't as helpful as we would have liked. It currently uses a weaker local LLM to save on costs, but there is an option to utilize the OpenAI API. Additionally, an improved prompt construction could help improve specificity and relevance
- Repair Plan usefulness: 4/5 - Although the repair plan takes a long time to generate, it provides it in easy to follow steps

Qualitative Results

- The system is able to capture subtle defects, and clearly outline and label the location of them in the image as well
- The system has a repair plan for each type of defect that is detected, making it easier to fix every type of defect that is detected.
- The defect detection still may miss some defects, or rarely incorrectly label defects. This can be improved on by increasing the quality of the dataset, or just improving the training model itself
- The chatbot can sometimes give vague advice, this may be due to the weaker LLM that is being used, the OpenAI API may give better results, however will cost money to use
- People may experience confusion when encountering missing data or unsupported defect types, recommending more explicit onboarding or in-interface tooltips

Implementation Details

System Hierarchy:

The system consists of comprehensive datasets containing information about defects and repairs such as detailed images and renders. Before the next step, if necessary, KiCad pcb models are created then exported to Blender. These datasets get fed into a virtual environment (Isaac Sim) for detection and repair training using a model. After training, the model is used to train the

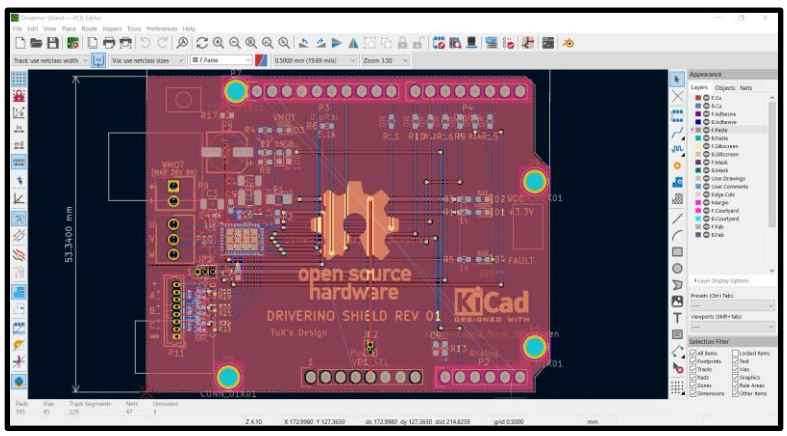
physical machine or is combined with other models to improve accuracy until requirements are met.

Individual Components & Tools:

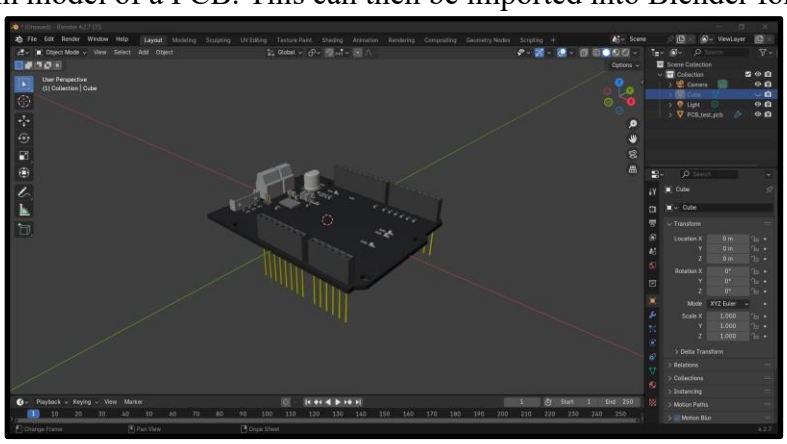
Virtual: images & renders (datasets), defect detection model, repair instructions model

Physical: machine, camera, sensors, linear control system, actuators

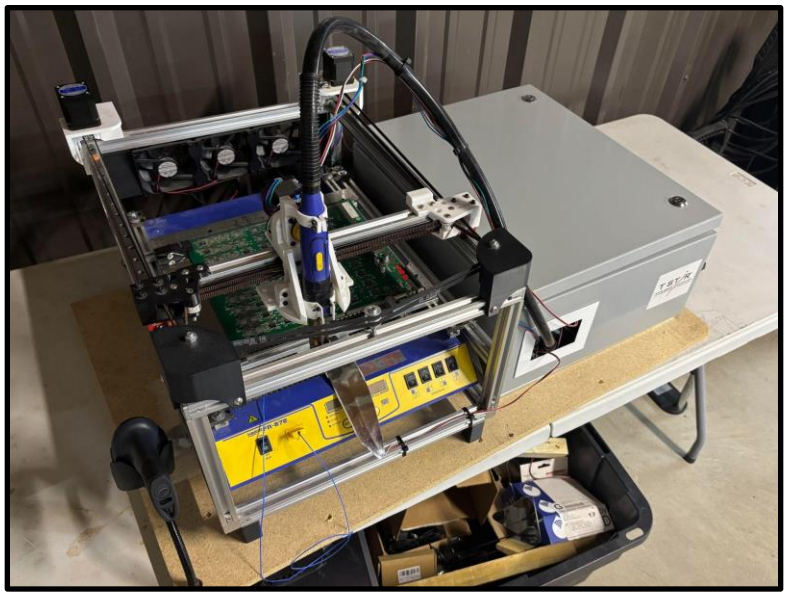
Project Imagery:



This is a KiCad PCB schematic of a DRIVERINO Arduino Shield, as an example, that constructs the main model of a PCB. This can then be imported into Blender for rendering.



The above is a Blender 3D version of the KiCad schematic seen previously. This will allow Blender to render out this PCB into 3-dimensional space. Defects can then be added for later training of an AI model (for common types of defects: faulty solder, board cracks, warping, etc.).



Seen above is an image of the physical PCB defect detection model prototype for this project. A virtual PCB defect detection environment (mentioned elsewhere in this document) will be the mind behind this physical model. Training AI through PCB datasets in the virtual environment will allow this hardware piece to effectively scan, detect, and repair a given PCB. In the image, the blue pen can desolder, remove, and re-solder components from the PCB in case of defect.

User Manual

Hardware Installation:

Jetson Orin - integrated camera - linear rails - flux dispenser - heating pen

Software Installation:

For Training: Install KiCad ver. 8.0 -> Install Blender ver. 4.2 + -> Install Isaac Sim

Operation Instructions:

For training: Create pcb models in KiCad following tutorial or guide. Once a .kicad_pcb file is completed, export as a .pc3d or .step file like so **File → Export**. These files will be imported into Blender through either the pcb2blender extension or STEPper add-on. Adding an extension /

add-on can be done through either the lookup library or manually installing add-on from the desktop.

Budget Costs

Internal Course Resource Needs

For our portion of the PCB repair system, the main resources we require are software and computing power to render images, train a LLM, and to run machine learning training and inference. If the hardware costs are too much, Cloud GPU compute instances have been considered as well for heavy training runs, having usage-based fees. The development environment will be built on open-source or free tools (Blender, NVIDIA Isaac Sim, YOLO, etc.) so the software costs will be minimal. The main expenses will come from high-performance computing resources that are necessary for training our detection models and running simulations in virtual environments. Depending on the amount of images generated for the synthetic dataset, some additional data storage may also be purchased. The industry mentor at Tuurny has offered a NVIDIA Jetson TX2 for our group to use, but to run stronger LLM models, it will likely not be powerful enough. Additionally, while physical computer vision and detection are a separate project given to another group, we plan to use a simpler camera setup just for our own testing purposes. This can be done by purchasing cheaper cameras, or through in-lab hardware.

Overall Cost Estimate for an Outside Company

Item	Description	Quantity	Stock Number	Unit Price	Total Price	Intended Purpose	Actual Usage
NVIDIA Jetson TX2 or stronger (currently have one)	Embedded computing device for running ML inference and simulation Pipelines	1	1	$598.00	$598.00	Generating synthetic data, local ML training	For data pipeline dev & training
Cloud GPU Compute Instances	GPU-enabled instances (AWS p2.xlarge or GCP equivalent)	~200 hours	N/A	$2.99/hr	~$300	Training ML models and running simulation	Estimated hours required for development

	for training and large-scale simulation run					workloads	
Additional Data Storage (S3/EBS, unsure if needed)	Storing large rendered images	~50GB for 2 months	N/A	~$0.023/GB/mo	$2.30	Retaining synthetic dataset outputs	Storing the rendered images generated from Blender

Discussion

Insights

Throughout the usability testing, our team gained valuable insight into how users with varying backgrounds interact with the *PCB Autonomy* system. The use of synthetic data for training the model and our choice to support both local and OpenAI-based LLM's provided users flexibility. Feedback revealed that OpenAI's GPT-4o was significantly faster and more responsive, enhancing the experience, especially for chatbot interactions.

The decision to include defect visualization via bounding boxes was also positively received and contributed to a better understanding of the summary. However, users unfamiliar with electronics suggested that repair steps could benefit from visual cues or simplified technology

Discussion

Team collaboration was crucial in addressing the challenges like long inference times and incorporating defect summaries meaningfully into the LLM prompt. If our team was to do the project again, we would adopt image-assisted prompts with OpenAI from the start, since that capability proved essential for detailed Q&A. Additionally, if there were more time, we would have liked to work with some physical prototype that our system could be incorporated into as well.

Reflection

Most of our proposed features worked as intended. Users found the system easy to navigate, and chatbot responses were marked as relevant by most people as well. Still, we hadn't initially accounted for users expecting chatbot availability before generating a repair plan, this caused confusion for some. In the future, we would redesign the chatbot interface to clarify this

dependency and visually indicate system readiness. Regarding the model to detect defects, we would continue adding to the dataset, and work on improving the mAP for it as well.

Future Work

There are several directions for future enhancements that emerged from our usability studies:

- **Incorporate visual aids in repair steps:** Users wanted visual instructions aligned with steps, especially those unfamiliar with PCB tools
- **Hardware prototype integration:** Though a physical prototype was planned, it remains a stretch goal. Tighter integration between software output and a robotic repair system would complete the automation pipeline
- **Image context in chat:** Future work should explore multimodal responses (GPT explaining defect locations directly)

Conclusion

The development of an autonomous robotic system capable of detecting, classifying, and repairing PCB defects describes a step toward intelligent electronics manufacturing and repair. By generating synthetic data and integrating with an LLM, success was achieved in creating a model with higher accuracy for interpretation of defects and generation of repair strategies. This allowed the system to detect issues and suggest viable repair solutions. This work creates a foundation for future improvements such as real-time operation and deployment on edge devices.

References

[1] Zhang, C., Shi, W., Li, X., Zhang, H., & Liu, H. (2018). Improved bare PCB defect detection approach based on deep feature learning. *IET Journal of Electronics*. https://doi.org/10.1049/joe.2018.8275

[2] Zang, S., Zhang, S., & Liu, S. (2024). *Research on PCB Defect Detection Method Based on Improved YOLOv8 Network. 408–412.* https://doi.org/10.1145/3703187.3703256

[3] Xin, H., Chen, Z., & Wang, B. (2021). PCB Electronic Component Defect Detection Method based on Improved YOLOv4 Algorithm. *Journal of Physics: Conference Series, 1827(1), 012167*. https://doi.org/10.1088/1742-6596/1827/1/012167

[4] Cherian, A., Abraham, J., Belhe, A., Tsai, O., & Brown, O. (2025). *Evaluating Large Language Models and CNNs for Industrial Defect Detection*. Unpublished manuscript. https://ieeexplore.ieee.org/abstract/document/10814831

[5] Jiang, W., Li, T., Zhang, S., Chen, W., & Yang, J. (2023). PCB defects target detection combining multi-scale and attention mechanism. *Engineering Applications of Artificial Intelligence, 123*, 106359. https://link.springer.com/article/10.1007/s40747-024-01554-5#Sec3

[6] X. Chen, Y. Wu, X. He and W. Ming, "A Comprehensive Review of Deep Learning-Based PCB Defect Detection," in IEEE Access, vol. 11, pp. 139017-139038, 2023, doi: 10.1109/ACCESS.2023.3339561. https://ieeexplore.ieee.org/document/10343144

[7] Chen, I.-C., Hwang, R.-C., & Huang, H.-C. (2023). PCB defect detection based on deep learning algorithm. Processes, 11(3), 775. https://www.mdpi.com/2227-9717/11/3/775

[8] Denninger, M., et al. (2020). *BlenderProc*: Procedural Generation of Photorealistic Scenes for AI-based Vision. DLR-RM. https://github.com/DLR-RM/BlenderProc

[9] Tobin, J., Biewald, L., Duan, E., Alex, P., Huan, G., & Abbeel, P**.** (2017). Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. https://arxiv.org/pdf/1710.06425

[10] Roy, A., Hasan, A., Ghosh, S., Varshney, N., Julia, J., Reza Forghani, & Navid Asadizanjani. (2024). Applications and Challenges of AI in PCB X-ray Inspection: A Comprehensive Study. ACM Journal on Emerging Technologies in Computing Systems. https://doi.org/10.1145/3703457

[11] Unexpected Maker. (2021, October 5). LIVE: Learning OpenCV - The Brute Force Method - PCB Inspection! YouTube. https://www.youtube.com/watch?v=SOb-ebO3yBA

[12] Voronova, A. (2024, January 3). *From KiCad to Blender for a Stunning Render*. Hackaday. Retrieved from https://hackaday.com/2024/01/03/from-kicad-to-blender-for-a-stunning-render/

[13] Antmicro. (2024, July 29). *Automated PCB Trace Selection in Signal Integrity Simulation for High-Speed Digital Interfaces*. Antmicro Blog. Retrieved from https://antmicro.com/blog/2024/07/automated-pcb-trace-selection-for-si-simulation/